

DevHub 通用社区程序改造：Codex 可执行任务文档

本文档用于交给 Codex / AI Agent 执行项目改造。

当前目标：将 DevHub 从“写死页面 Demo”升级为“通用社区程序 + 默认页面主题”。

注意：网站名称始终保持为 **DevHub**，不要改成 LearnKu、LearnKu Clone 或其他名称。

1. 项目定位

1.1 当前改造目标

DevHub 不再只是一个固定的技术社区页面，而是要升级成一个可配置、可扩展、可复用的通用社区程序。

新的定位：

DevHub = 通用社区程序核心能力 + 默认技术社区主题页面

也就是说：

页面部分：负责展示、布局、交互、主题风格

程序功能：负责社区、板块、内容、用户、评论、搜索、权限、运营等核心能力

1.2 核心原则

1. 页面不写死业务数据。
2. PHP / Go / Java 只是默认初始化数据，不是程序固定结构。
3. 社区 / 问答中心 / 开源项目 / AI作品 / 招聘内推 / Wiki / 文档 只是默认板块，不是程序固定结构。
4. 程序要支持后续创建更多站点、更多板块、更多内容类型。
5. 页面主题可以替换，程序功能是核心资产。
6. DevHub 默认是技术社区，但底层能力应能复用于其他社区，例如美业社区、电商社区、企业知识社区、开源项目社区等。

2. 拆分后的整体结构

2.1 两个大部分

需要将当前项目拆成两个明确部分：

DevHub

└── 页面层 / 前端主题

└── 程序功能层 / 后端社区系统

2.2 页面层职责

页面层负责：

1. 页面布局
2. 顶部导航
3. 子站切换
4. 搜索框
5. 左侧分类
6. 右侧标签合集
7. 内容列表展示
8. 内容详情展示
9. 评论展示
10. 用户中心展示
11. 管理后台展示
12. 前端路由和接口调用

页面层不负责：

1. 决定有哪些社区
2. 决定有哪些板块
3. 决定用户能不能发帖
4. 决定内容是否需要审核
5. 决定积分规则
6. 决定搜索范围
7. 决定权限规则

这些规则都应交给后端程序功能层。

2.3 程序功能层职责

程序功能层负责：

1. 社区 / 子站管理
2. 板块管理
3. 内容发布与管理
4. 标签管理
5. 评论管理
6. 用户系统
7. 角色权限
8. 搜索
9. Feed 动态流

10. 点赞 / 收藏 / 关注
11. 积分
12. 通知
13. 审核
14. 系统配置
15. 管理后台接口

3. 当前版本执行目标

3.1 当前优先版本：v0.1

v0.1 不追求完整社区系统，而是先完成“通用社区程序骨架”。

v0.1 目标：

1. DevHub 可以正常运行
2. 前端页面不再写死 PHP / Go / Java 数据
3. 后端提供 site / board / content / tag / comment / search 基础接口
4. 页面通过接口渲染社区、板块、内容、标签
5. 支持总站和子站两种浏览模式
6. 支持总站搜索和子站搜索
7. 支持最小内容详情页
8. 支持最小评论展示

3.2 v0.1 必须完成的模块

1. site 社区模块
2. board 板块模块
3. content 内容模块
4. tag 标签模块
5. comment 评论模块
6. search 搜索模块
7. 默认初始化数据
8. 默认页面主题接入

3.3 v0.1 暂时不强制完成的模块

以下模块可以预留结构，但不要作为 v0.1 的阻塞项：

1. 完整登录注册
2. JWT 双 Token
3. 完整用户中心
4. 点赞
5. 收藏

- 6. 关注
- 7. 积分
- 8. 通知
- 9. 审核后台
- 10. 完整 RBAC 权限
- 11. 全文搜索引擎
- 12. 推荐算法

4. 页面部分拆分要求

4.1 页面层目录建议

如果当前项目已经有前端目录，请尽量在现有结构中调整，不要无意义大规模重建。推荐结构如下：

```
frontend/
├── pages/
│   ├── home/           # 总站首页
│   ├── site/           # 子站首页
│   ├── board/          # 板块页
│   ├── content/        # 内容详情页
│   ├── search/         # 搜索结果页
│   ├── user/           # 用户中心，后续实现
│   └── admin/          # 管理后台，后续实现
├── components/
│   ├── layout/         # 公共布局
│   ├── site-switcher/  # 子站切换
│   ├── search-box/     # 搜索框
│   ├── content-card/   # 内容卡片
│   ├── tag-list/       # 标签合集
│   ├── board-nav/      # 板块导航
│   ├── comment-list/   # 评论列表
│   └── user-menu/      # 用户菜单
├── api/                # 前端 API 封装
├── store/              # 状态管理，可选
└── styles/             # 样式
```

如果当前是单 HTML Demo，则先按以下方式拆分逻辑：

1. 保留当前可运行页面
2. 去掉页面中的写死数据
3. 增加 API 请求层
4. 页面启动时请求 `/api/v1/sites`
5. 根据接口返回渲染总站、子站、板块、标签、内容列表

4.2 页面清单

4.2.1 总站首页

路由示例：

/

展示内容：

1. DevHub 顶部导航
2. 子站切换入口
3. 搜索框
4. 全站热门
5. 最新内容
6. 推荐社区
7. 推荐标签
8. 推荐问答
9. 推荐开源项目
10. 推荐文章 / Wiki / 文档

规则：

1. 不限定 site_key
2. 展示全部社区内容集合
3. 搜索时默认全站搜索

4.2.2 子站首页

路由示例：

/sites/:site_key

例如：

/sites/php
/sites/go
/sites/java

展示内容：

1. 当前子站名称
2. 当前子站 logo
3. 当前子站描述

4. 当前子站板块导航
5. 当前子站热门内容
6. 当前子站最新帖子
7. 当前子站标签合集

规则：

1. 页面只展示当前 `site_key` 下的内容
2. 搜索时默认只搜索当前子站内容
3. 点击子站 logo 默认回到当前子站首页
4. 点击 DevHub 总站 logo 回到总站首页

4.2.3 板块页

路由示例：

```
/sites/:site_key/boards/:board_key
```

展示内容：

1. 板块标题
2. 板块描述
3. 板块类型
4. 筛选条件
5. 排序方式
6. 内容列表
7. 分页

规则：

1. 必须同时按 `site_key` 和 `board_key` 过滤
2. 支持按最新、热门、评论数排序
3. 后续支持按标签筛选

4.2.4 内容详情页

路由示例：

```
/contents/:id
```

展示内容：

1. 标题
2. 作者

3. 发布时间
4. 所属社区
5. 所属板块
6. 标签
7. 正文
8. 浏览数
9. 点赞数
10. 收藏数
11. 评论列表
12. 相关推荐

v0.1 要求：

1. 能正常展示详情
2. 能展示评论列表
3. 点赞、收藏按钮可以先做 UI，不强制实现真实功能

4.2.5 搜索结果页

路由示例：

```
/search?keyword=xxx  
/search?keyword=xxx&site_key=php
```

展示内容：

1. 搜索关键词
2. 当前搜索范围
3. 子站筛选
4. 板块筛选
5. 标签筛选
6. 排序方式
7. 搜索结果列表

搜索规则：

1. 没有 site_key: 全站搜索
2. 有 site_key: 当前子站搜索
3. 有 board_key: 当前板块搜索
4. 有 tag: 当前标签搜索

4.2.6 用户中心页面，v0.2 实现

预留页面：

```
/user/profile  
/user/contents  
/user/comments  
/user/collections  
/user/points  
/user/notifications
```

4.2.7 管理后台页面，v0.4 实现

预留页面：

```
/admin  
/admin/sites  
/admin/boards  
/admin/contents  
/admin/users  
/admin/tags  
/admin/comments  
/admin/audit  
/admin/settings
```

5. 程序功能模块拆分

后端按功能模块拆，不要按页面拆。

5.1 后端目录建议

如果当前项目已经有目录结构，请尽量保留并逐步重构。推荐结构：

```
backend/  
├── cmd/  
│   └── server/  
├── internal/  
│   ├── auth/  
│   ├── user/  
│   ├── site/  
│   ├── board/  
│   ├── content/  
│   ├── comment/  
│   ├── tag/  
│   ├── search/  
│   ├── feed/  
│   ├── like/  
│   ├── collect/  
│   └── follow/
```



```
|   |—— point/
|   |—— notification/
|   |—— audit/
|   |—— role/
|   |—— config/
|—— pkg/
|—— configs/
|—— migrations/
```

v0.1 先实现：

```
internal/site
internal/board
internal/content
internal/tag
internal/comment
internal/search
```

其他模块可以只预留目录或 TODO。

6. 核心数据模型

6.1 site：社区 / 子站

用途：表示一个独立社区或子站。

示例数据：

```
php
go
java
ai
ecommerce
```

字段建议：

```
site
|—— id                bigint
|—— site_key          varchar, 唯一, 例如 php / go / java
|—— name              varchar, 例如 PHP 社区
|—— logo              varchar
|—— description        varchar/text
|—— domain            varchar, 可为空
|—— status            tinyint, 1 启用, 0 禁用
|—— sort              int
```

— config	json
— create_time	bigint
— update_time	bigint
— delete_time	bigint

6.2 board：板块

用途：表示某个社区下面的板块。

示例数据：

社区
问答中心
开源项目
AI作品
招聘内推
Wiki
文档

字段建议：

board	
— id	bigint
— site_id	bigint
— board_key	varchar, 例如 community / questions / projects / ai-works
— name	varchar
— description	varchar/text
— type	varchar, 例如 post / question / project / work / job / wiki / doc
— icon	varchar
— status	tinyint, 1 启用, 0 禁用
— sort	int
— config	json
— create_time	bigint
— update_time	bigint
— delete_time	bigint

board.type 推荐枚举：

post	普通帖子
question	问答
article	文章
project	开源项目
work	AI作品 / 作品展示
job	招聘
wiki	Wiki
doc	文档

6.3 content：内容

用途：统一承载帖子、问答、文章、项目、Wiki、文档、招聘、作品等内容。

字段建议：

content	
— id	bigint
— site_id	bigint
— board_id	bigint
— user_id	bigint
— content_type	varchar
— title	varchar
— summary	varchar/text
— body	longtext
— cover	varchar
— status	tinyint
— view_count	int
— like_count	int
— comment_count	int
— collect_count	int
— is_top	tinyint
— is_recommend	tinyint
— is_essence	tinyint
— publish_time	bigint
— create_time	bigint
— update_time	bigint
— delete_time	bigint

content_type 推荐枚举：

```
post
question
article
project
work
job
wiki
doc
```

content.status 推荐枚举：

```
0 草稿
1 已发布
2 待审核
3 审核拒绝
```

- 4 已隐藏
- 5 已删除

6.4 tag: 标签

用途：支持全站标签和子站标签。

字段建议：

```
tag
├── id          bigint
├── site_id     bigint, 0 表示全站标签
├── name        varchar
├── slug        varchar
├── description  varchar/text
├── use_count   int
├── status      tinyint
├── create_time bigint
├── update_time bigint
└── delete_time bigint
```

6.5 content_tag: 内容标签关系

```
content_tag
├── content_id  bigint
└── tag_id     bigint
```

6.6 comment: 评论

用途：支持内容评论和二级回复。

字段建议：

```
comment
├── id          bigint
├── content_id  bigint
├── user_id     bigint
├── parent_id   bigint, 0 表示一级评论
├── reply_user_id bigint, 可为空
├── body        text
├── status      tinyint
├── like_count  int
├── create_time bigint
├── update_time bigint
└── delete_time bigint
```

comment.status 推荐枚举：

- 1 正常
- 2 待审核
- 3 已隐藏
- 4 已删除

6.7 user：用户，v0.2 实现

v0.1 可以先使用 mock 用户。

字段建议：

```
user
├── id
├── username
├── nickname
├── avatar
├── email
├── phone
├── password_hash
├── status
├── last_login_time
├── create_time
├── update_time
└── delete_time
```

7. API 接口设计

7.1 API 基础约定

统一前缀：

```
/api/v1
```

统一返回结构：

```
{
  "code": 0,
  "message": "success",
  "data": {}
}
```

分页返回结构：

```
{
  "code": 0,
  "message": "success",
  "data": {
    "list": [],
    "page": 1,
    "page_size": 20,
    "total": 0
  }
}
```

错误返回示例：

```
{
  "code": 40001,
  "message": "参数错误",
  "data": null
}
```

7.2 健康检查

```
GET /api/v1/health
```

返回：

```
{
  "code": 0,
  "message": "success",
  "data": {
    "status": "ok",
    "name": "DevHub"
  }
}
```

7.3 site 接口

获取社区列表

```
GET /api/v1/sites
```

用途：页面启动时获取所有启用的社区。

返回字段：

```
id
site_key
name
logo
description
status
sort
```

获取社区详情

```
GET /api/v1/sites/:site_key
```

用途：进入子站首页时获取当前社区信息。

7.4 board 接口

获取某社区下的板块列表

```
GET /api/v1/sites/:site_key/boards
```

用途：子站首页和板块导航使用。

获取板块详情

```
GET /api/v1/sites/:site_key/boards/:board_key
```

用途：进入板块页时获取板块信息。

7.5 content 接口

获取内容列表

```
GET /api/v1/contents
```

查询参数：

site_key	可选
board_key	可选
content_type	可选
tag	可选
keyword	可选
sort	可选, latest / hot / comments / views

page	默认 1
page_size	默认 20

规则：

1. 没有 site_key: 查全站内容
2. 有 site_key: 查指定社区内容
3. 有 board_key: 查指定板块内容
4. 有 content_type: 查指定内容类型
5. 有 keyword: 按标题、摘要、正文搜索

获取内容详情

GET /api/v1/contents/:id

要求：

1. 返回内容基础信息
2. 返回所属 site 信息
3. 返回所属 board 信息
4. 返回作者信息
5. 返回标签列表
6. 浏览数可以在此接口中递增，也可以后续独立实现

创建内容，v0.2 可真实实现

POST /api/v1/contents

v0.1 可以先保留接口结构，允许 mock 创建或返回未登录提示。

请求体示例：

```
{
  "site_key": "php",
  "board_key": "questions",
  "content_type": "question",
  "title": "PHP 如何优化 Composer 加载速度？",
  "summary": "想了解 Composer 自动加载优化方式",
  "body": "正文内容",
  "tags": ["PHP", "Composer", "性能优化"]
}
```


7.6 tag 接口

获取标签列表

```
GET /api/v1/tags
```

查询参数：

```
site_key 可选  
limit    可选
```

规则：

1. 没有 site_key: 返回全站热门标签
2. 有 site_key: 返回当前社区热门标签

7.7 comment 接口

获取内容评论

```
GET /api/v1/contents/:id/comments
```

要求：

1. 返回一级评论
2. 返回二级回复
3. 按 create_time 正序或热度排序

创建评论，v0.2 可真实实现

```
POST /api/v1/contents/:id/comments
```

v0.1 可以先保留接口结构。

7.8 search 接口

```
GET /api/v1/search
```

查询参数：

```
keyword
site_key
board_key
content_type
tag
sort
page
page_size
```

搜索规则：

1. 没有 site_key: 全站搜索
2. 有 site_key: 当前子站搜索
3. 有 board_key: 当前板块搜索
4. 有 tag: 当前标签搜索

v0.1 可以先使用数据库 like 查询或内存过滤。后续再接入全文搜索。

8. 默认初始化数据

v0.1 必须提供默认初始化数据，确保项目启动后页面有内容可展示。

8.1 默认社区

1. PHP 社区
site_key: php
description: PHP、Laravel、Hyperf、Composer、后端工程化交流社区
2. Go 社区
site_key: go
description: Go、Gin、微服务、云原生、工程实践交流社区
3. Java 社区
site_key: java
description: Java、Spring、JVM、分布式系统交流社区

8.2 每个社区默认板块

每个社区默认创建以下板块：

1. 社区
board_key: community
type: post

2. 问答中心
board_key: questions
type: question
3. 开源项目
board_key: projects
type: project
4. AI作品
board_key: ai-works
type: work
5. 招聘内推
board_key: jobs
type: job
6. Wiki
board_key: wiki
type: wiki
7. 文档
board_key: docs
type: doc

8.3 默认内容

每个社区至少创建 3 到 5 条内容，覆盖不同板块。

示例：

PHP 社区：

1. Hyperf 项目目录结构最佳实践
2. Composer 自动加载优化经验
3. Laravel 和 Hyperf 的依赖注入差异

Go 社区：

1. Gin 项目如何拆分路由和服务层
2. Go 项目中如何管理配置文件
3. 使用 GORM 设计通用社区模型

Java 社区：

1. Spring Boot 项目模块拆分建议
2. JVM 参数调优入门
3. MyBatis 和 JPA 的使用边界

8.4 默认标签

示例：

PHP
Hyperf
Laravel
Composer
Go
Gin
GORM
Java
Spring Boot
JVM
MySQL
Redis
Docker
AI
开源项目
性能优化
架构设计

9. 分阶段实现路线

9.1 v0.1：通用社区骨架

必须完成：

1. site 模块
2. board 模块
3. content 模块
4. tag 模块
5. comment 模块
6. search 模块
7. 默认初始化数据
8. 前端页面改为接口驱动
9. 总站首页
10. 子站首页
11. 板块页
12. 内容详情页
13. 搜索结果页

验收标准：

1. 项目可以正常启动
2. 打开首页能看到 DevHub 总站
3. 能看到 PHP / Go / Java 子站切换
4. 点击子站后只展示当前子站内容
5. 点击板块后只展示当前板块内容

6. 点击内容能进入详情页
7. 详情页能展示标签和评论
8. 总站搜索能搜索全部内容
9. 子站搜索只搜索当前子站内容
10. 页面中不再硬编码 PHP / Go / Java 内容数据

9.2 v0.2: 用户与内容互动

后续完成:

1. 注册
2. 登录
3. 发帖
4. 评论
5. 点赞
6. 收藏
7. 用户中心
8. JWT / Token 鉴权

9.3 v0.3: 社区运营能力

后续完成:

1. 关注
2. 积分
3. 通知
4. 热门内容
5. 推荐内容
6. 精华
7. 置顶
8. Feed 动态流

9.4 v0.4: 管理后台

后续完成:

1. 社区管理
2. 板块管理
3. 内容管理
4. 用户管理
5. 标签管理
6. 评论管理
7. 审核管理
8. 系统设置

10. Codex 执行要求

10.1 执行前先检查项目

Codex 执行时，先做以下检查：

1. 查看项目目录结构
2. 判断当前是单 HTML Demo、前后端混合项目，还是已有后端项目
3. 保留当前能运行的功能
4. 不要一次性推翻重写，除非当前结构已经完全不可维护
5. 优先完成 v0.1 的可运行闭环

10.2 不要做的事情

本阶段不要做：

1. 不要把网站名称改成 LearnKu
2. 不要把 PHP / Go / Java 写死在前端页面里
3. 不要一开始就实现复杂权限系统
4. 不要一开始就接入 Elasticsearch / Meilisearch 等搜索引擎
5. 不要一开始就做复杂推荐算法
6. 不要一开始就做完整后台 UI
7. 不要破坏当前页面已有的视觉结构
8. 不要为了重构而重构

10.3 优先做的事情

本阶段优先做：

1. 抽象 site / board / content / tag / comment 模型
2. 提供 API
3. 提供默认数据
4. 让页面通过 API 渲染
5. 保证总站、子站、板块、详情、搜索可用
6. 保证项目能本地运行

10.4 技术实现建议

如果项目已经选定技术栈，则沿用当前技术栈。

如果当前后端为 Go，可优先使用：

Go + Gin + MySQL 8

如果当前还没有数据库，可先使用内存数据跑通 v0.1，再补 MySQL 迁移。

推荐顺序：

第一步：内存数据跑通接口
第二步：页面改成请求接口
第三步：再接 MySQL 8
第四步：补充迁移文件和初始化数据

不要为了数据库设计拖慢页面和接口闭环。

11. 推荐接口清单汇总

v0.1 必须实现：

```
GET /api/v1/health
GET /api/v1/sites
GET /api/v1/sites/:site_key
GET /api/v1/sites/:site_key/boards
GET /api/v1/sites/:site_key/boards/:board_key
GET /api/v1/contents
GET /api/v1/contents/:id
GET /api/v1/contents/:id/comments
GET /api/v1/tags
GET /api/v1/search
```

v0.2 再实现：

```
POST /api/v1/auth/register
POST /api/v1/auth/login
GET /api/v1/auth/me
POST /api/v1/auth/logout
POST /api/v1/contents
POST /api/v1/contents/:id/comments
POST /api/v1/contents/:id/like
POST /api/v1/contents/:id/collect
```

v0.3 再实现：

```
GET /api/v1/feed/latest
GET /api/v1/feed/hot
GET /api/v1/feed/recommend
POST /api/v1/follows
```

```
GET /api/v1/notifications
GET /api/v1/points/logs
```

v0.4 再实现：

```
GET /api/v1/admin/sites
POST /api/v1/admin/sites
PUT /api/v1/admin/sites/:id
DELETE /api/v1/admin/sites/:id

GET /api/v1/admin/boards
POST /api/v1/admin/boards
PUT /api/v1/admin/boards/:id
DELETE /api/v1/admin/boards/:id

GET /api/v1/admin/contents
PUT /api/v1/admin/contents/:id/status

GET /api/v1/admin/users
PUT /api/v1/admin/users/:id/status

GET /api/v1/admin/comments
PUT /api/v1/admin/comments/:id/status

GET /api/v1/admin/settings
PUT /api/v1/admin/settings
```

12. 前端交互规则

12.1 子站切换

1. 顶部显示 DevHub 总站入口
2. 顶部显示子站切换入口
3. 子站数据来自 GET /api/v1/sites
4. 点击 DevHub logo 回到 /
5. 点击某个子站进入 /sites/:site_key
6. 进入子站后，页面搜索默认带上 site_key

12.2 搜索框规则

1. 在总站首页搜索：跳转 /search?keyword=xxx
2. 在子站页面搜索：跳转 /search?keyword=xxx&site_key=当前site_key
3. 搜索框需要展示当前搜索范围
4. 支持用户手动切换搜索范围：全站 / 当前子站

12.3 内容卡片规则

内容卡片至少展示：

1. 标题
2. 摘要
3. 所属社区
4. 所属板块
5. 作者
6. 发布时间
7. 标签
8. 浏览数
9. 评论数
10. 点赞数

12.4 右侧标签合集

1. 总站显示全站热门标签
2. 子站显示当前子站热门标签
3. 点击标签进入搜索结果页
4. 总站标签搜索不带 site_key
5. 子站标签搜索带 site_key

13. 数据边界规则

13.1 总站

总站可以展示全部内容：

site_key 为空

适用页面：

/
/search

13.2 子站

子站只展示当前 site 下的数据：

```
site_key = php / go / java / ...
```

适用页面：

```
/sites/:site_key  
/sites/:site_key/boards/:board_key  
/search?site_key=:site_key
```

13.3 板块

板块必须依赖社区：

```
site_key + board_key
```

不要只用 board_key 查询，因为不同社区可能存在相同 board_key。

14. 数据库设计原则

1. 使用 MySQL 8 时，json 字段可以使用 JSON 类型。
2. 所有表建议保留 create_time、update_time、delete_time。
3. 时间字段可以使用 bigint Unix 时间戳，保持当前项目风格统一。
4. 业务删除优先使用软删除 delete_time。
5. 列表查询默认只查 status 正常且 delete_time = 0 的数据。
6. site_key、board_key、slug 等字段需要建立索引。
7. content 表需要按 site_id、board_id、content_type、status、publish_time 建索引。

15. 建议的开发顺序

Codex 请按照以下顺序执行：

1. 检查当前项目结构和运行方式
2. 确认 DevHub 当前页面入口
3. 增加或整理后端 API 基础结构
4. 实现 /api/v1/health
5. 实现 site 模块和默认数据
6. 实现 board 模块和默认数据
7. 实现 content 模块和默认数据
8. 实现 tag 模块和默认数据
9. 实现 comment 模块和默认数据
10. 实现 search 查询
11. 修改前端页面，从接口读取 sites
12. 修改前端页面，从接口读取 boards

13. 修改前端页面，从接口读取 contents
14. 修改搜索逻辑，区分总站搜索和子站搜索
15. 修改详情页，读取 content detail 和 comments
16. 确保本地启动正常
17. 提供运行命令和测试命令

16. 最小验收用例

16.1 后端验收

执行以下请求应返回正常数据：

```
curl http://127.0.0.1:8080/api/v1/health
curl http://127.0.0.1:8080/api/v1/sites
curl http://127.0.0.1:8080/api/v1/sites/php
curl http://127.0.0.1:8080/api/v1/sites/php/boards
curl "http://127.0.0.1:8080/api/v1/contents?site_key=php"
curl "http://127.0.0.1:8080/api/v1/search?keyword=Hyperf&site_key=php"
```

16.2 前端验收

页面应满足：

1. 首页能打开
2. 顶部显示 DevHub
3. 能看到 PHP / Go / Java 子站
4. 点击 PHP 子站，只展示 PHP 内容
5. 点击 Go 子站，只展示 Go 内容
6. 点击 Java 子站，只展示 Java 内容
7. 点击板块，只展示当前子站当前板块内容
8. 点击内容卡片，进入详情页
9. 搜索框在总站搜索全站
10. 搜索框在子站搜索当前子站
11. 右侧标签合集能根据总站 / 子站变化

17. 最终结果要求

完成后，请输出以下内容：

1. 修改了哪些文件
2. 新增了哪些模块
3. 新增了哪些接口

- 4. 当前如何启动项目
- 5. 当前如何测试接口
- 6. 哪些功能已经完成
- 7. 哪些功能只是预留
- 8. 下一步建议做什么

18. 一句话总结

DevHub 要从“固定 PHP / Go / Java 技术站页面”升级为“可配置的通用社区程序”。

当前阶段不要追求大而全，先完成：

site + board + content + tag + comment + search + 默认页面主题

只要这个闭环跑通，DevHub 就具备了后续扩展成完整社区系统的基础。